Shallow Neural Networks for Function Approximation and Solving Boundary Value Problems

University of Birmingham

Anastasia Istratuca

Supervisor: Dr. Daniel Loghin

13 September 2019

Contents

1 Abstract **4**

2 Introduction **5**

3 Background **6**
3.1 Approximation Theory 6
3.2 The Weierstrass Theorem 6
3.3 Polynomial interpolation 6
3.4 Problems with polynomial interpolation 7
3.5 Least-squares fitting 9

4 Neural Networks **11**

5 Numerical experiments **14**
5.1 Initial guess 14
5.2 Optimisation 14
5.3 Numerical results in 1D 14
5.4 Numerical results in multiple dimensions 16
5.5 Problems with Neural Network Approximation 17
5.6 Numerical results - Piecewise Neural Networks 18
5.7 Problems with piecewise approach 21

6 Boundary Value Problems **22**
6.1 The Variational and Weak Formulations 22
6.2 Trial Functions and Neural Network Training 22
6.3 Numerical Results 23

7 Summary **26**

List of Figures

1 Polynomial interpolation of error function 7
2 Polynomial interpolation of $f(x) = e^x$ 8
3 Polynomial interpolation of $f(x) = e^{-x} \sin x$ 8
4 Polynomial interpolation of Bessel's function of the first kind 8
5 Polynomial interpolation on equidistant nodes of Runge's function . . 9
6 Polynomial interpolation on Chebyshev nodes of Runge's function . . 10
7 Deep Neural Network with 5 layers. 12
8 Shallow neural network approximation of error function 15
9 Shallow neural network approximation of $f(x) = e^x$ 15
10 Shallow neural network approximation of $f(x) = e^{-x} \sin x$ 16
11 Shallow neural network approximation of Bessel's function (1st kind) 16
12 Neural network approximation on equidistant nodes of Runge's function 16
13 Neural network approximation on Chebyshev nodes of Runge's function 17
14 Functions of two variables shallow neural network approximation . . . 17
15 Shallow neural network approximation of $\log(|x| + \frac{1}{e})$ 18
16 Piecewise neural network approximation of error function 19
17 Piecewise neural network approximation of $f(x) = e^x$ 19
18 Piecewise neural network approximation of $f(x) = e^{-x} \sin x$ 19
19 Piecewise neural network approximation of Bessel's function (1st kind) 20
20 Piecewise neural network approximation of Runge's function 20
21 Piecewise neural network approximation of $f(x) = \log(|x| + \frac{1}{e})$ 20
22 Deep Neural Network solution of Boundary Value Problem 6.4 24
23 Deep Neural Network solution of Boundary Value Problem 6.5 25

1 Abstract

This report presents the results obtained using neural networks for approximating functions and boundary value problems (BVP's). The method implements the Levenberg Marquardt algorithm to solve an unconstrained minimisation approach for function approximation. It further applies the Interior-point method to solve a constrained minimisation problem for solving PDE's.

Compared to previous numerical methods used in function approximation, such as Lagrange interpolation or rational approximation, the neural network approach seems to produce better results with less parameters and the obtained approximation can be extrapolated with a fairly small error. Additionally, inspired by the piecewise polynomial approximation method, this procedure can also be adapted to a piecewise neural network approximation.

For the case of boundary value problems, the cost function used for training the network is based on the variational formulation. One advantage of the deep learning approach together with the Ritz method is the ability to solve non-linear problems, at least in one dimension. Another benefit of this approach is that it is highly adaptable. In particular, increasing the number of dimensions of the input and transitioning from linear equations to non-linear ones is reasonably easy.

This report does not present any novel material. Instead, it aims to verify some already known theoretical results and to highlight some problems we encountered in experiments.

2 Introduction

Function approximation has been extensively studied and several numerical methods, such as Lagrange interpolation or Least Squares fitting, discussed in the following section, have already been developed for solving this problem. Additionally, numerical methods such as the finite difference method (Smith 1986; Mazumder 2015) or finite element method (Strang and Fix 2008; Johnson 2009), have also been established for solving partial differential equations numerically (Morton and Mayers 2005). However, one disadvantage of these methods is that they tend to be difficult, nearly impossible, to implement in a very large number of dimensions.

Artificial neural networks (ANNs), deep learning in particular, have shown encouraging results in various fields, such as image classification, speech and text recognition, natural language processing and generating data. Mathematically, deep learning is based on a new way to approximate functions, which naturally led to their application in other areas that involve constructing functions, such as the ones presented in this report.

The material covers the basics of approximation theory, with an emphasis on the Lagrange interpolation and Least squares methods. This section also describes some of the problems with these approaches as a motivation for using neural networks.

The following section introduces the fundamental concepts behind artificial neural networks. It defines an ANN, from neurons and layers, weights and biases to activation functions. Furthermore, it details the training process, including the definition of a cost function and the purpose of an optimisation algorithm. The section also briefly touches on the Universal Approximation Theory.

Next, the artificial neural network approach is used to approximate various functions, with focus on the error function, the exponential and Runge's function in one dimension. Some numerical results are presented, including the ones obtained from the piecewise approach on the same functions. The method is extended to functions of two variables and it can be further expanded to functions of any number of variables.

The deep learning approach for boundary value problems is based on the Deep Ritz Method (E and Yu 2018). It is illustrated on Poisson's equation and some of its non-linear variations subject to boundary conditions in one dimension. Additionally, the method has the potential to overcome the "curse of dimensionality".

Lastly, an argument on why this approach is superior to traditional numerical methods is given. Furthermore, some problems encountered during the experiments are described, which could be the subject of further investigations.

3 Background

3.1 Approximation Theory

Definition 3.1 (*Approximation theory*)**.** Approximation theory is the branch of mathematics concerned with how functions can be approximated with simpler, more suitable functions.

The notion of *best* or *simplest* approximation varies and it depends on the formulation used, for example on the choice of function spaces or the norms used.

The study of approximation theory covers two general types of problems. In the first problem, the function f we are trying to approximate is given explicitly and we wish to replace it with a simpler function g. The second problem involves fitting functions to given data. In other words, instead of an explicit formula, only a set of points $\{(x_i, f(x_i)\}_{i=0}^{i=n}$ is provided and we want to find the best function g to represent it.

There are two principal uses for function approximation, which correspond to the two types of problems mentioned above. Firstly, replacing a complicated looking function by a simpler one, many common operations such as differentiation, integration and even evaluation become easier to perform. For example, $\ln(x)$, $\sin(x)$ or the error function cannot be evaluated using basic arithmetic operations without first approximating them. Secondly, suppose the values of the function $f(x)$ are known for only a finite, normally small, number of points $\{x_i\}_{i=0}^{i=n}$. Function approximation allows us to generate values of $f(x)$, for all points x in the approximating interval, where $x \neq x_i$, $i = 0, \ldots, n$.

The choice of these simpler functions used in approximation theory, let us call them base functions, is still being researched. Some well-known options are polynomials, ratios of polynomials, sines and cosines.

For the purpose of this report, we will focus on the problem of approximating a function using operations that can be performed on a computer, namely arithmetic operations and some logical operations, such as algebraic comparison. This is typically done using polynomial approximations.

3.2 The Weierstrass Theorem

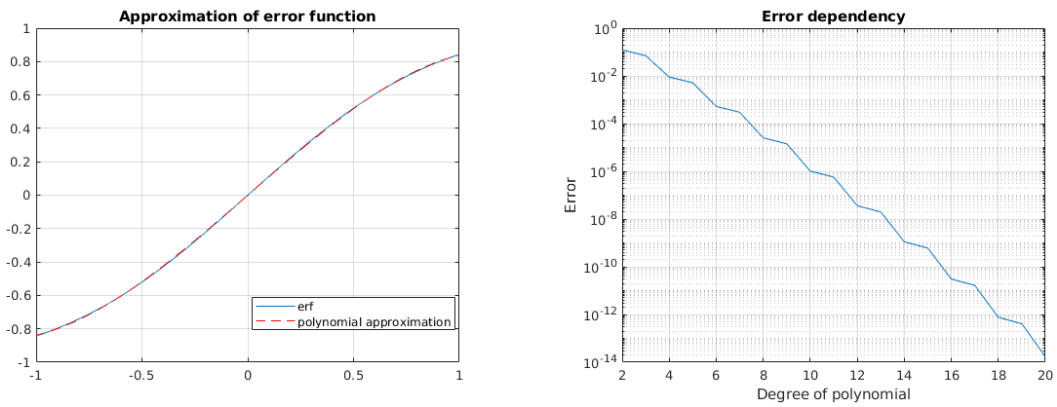
Theorem 3.1 (*Weierstrass Theoreom* (Cheney 1966))**.** Let f be a continuous function defined on $[a, b]$. Then for all $\epsilon > 0$, $\exists$ a polynomial P such that:

$$\max \mid f(x) - P(x) \mid < \epsilon, \forall x \in [a, b].$$

Weierstrass' Theorem confirms that there exists a polynomial p that approximates f with arbitrarily precision. However, it does not provide any additional information about it, such as the degree of the polynomial p or whether it is unique or not. Next, we focus on the technique of polynomial interpolation, using monomials as a basis for the interpolant.

3.3 Polynomial interpolation

We continue the current section by stating an important result, continue with mathematical analysis and finish with examples. To this purpose, we define Π_n to be the space of polynomials of degree n.



(a) Approximation using $p \in \Pi_4$ (b) Error dependence on n

Figure 1: Polynomial interpolation of error function

Theorem 3.2. *Theorem of Polynomial Interpolation* (Kincaid and Cheney 2009) If $\{x_i\}_{i=0}^{i=n}, i = 0, \ldots, n$ are distinct real numbers, then for arbitrary values $\{y_i\}_{i=0}^{i=n}$, $i = 0, \ldots n$, there is a unique polynomial p_n of degree at most n such that:

$$p_n(x_i) = y_i, \quad i = 0, \ldots, n.$$

Hence, combining this result with Weirestrass' Theorem, if we have $n+1$ points, and we know the value of the function we are trying to approximate at those points, we can always construct a polynomial which estimates fairly well the initial function.

Suppose we have a set of distinct points and the values of the function $f_i := f(x_i)$, $i = 0, \ldots, n$ at those points $\{x_i, f_i\}$, $i = 0, \ldots, n$. Our aim is to construct a polynomial $P(x_i)$, such that $P(x_i) = f_i$, $i = 0, \ldots, n$. Now, $P(x_i)$ can be written as:

$$\sum_{j=0}^{n} a_j x_i^j = f_i, \quad i = 0, \ldots, n.$$

If we rearrange in matrix form, we get:

$$\begin{bmatrix} 1 & x_0 & \ldots & x_0^n \\ \vdots & \ddots & \ddots & \vdots \\ 1 & x_n & \ldots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} f_0 \\ \vdots \\ f_n \end{bmatrix}$$

The matrix on the left hand side is known as the *Vandermonde* matrix and it has the property that its determinant is non-zero if $x_i \neq x_j$, $\forall i \neq j$, which is the case in the equation above. Hence, the system has a unique solution, given by:

$$\underline{a} = V^{-1}\underline{f},$$

where $\underline{a}$ is the vector containing the coefficients of the approximating polynomial.

Using MATLAB, we implemented this algorithm and the results can be seen in Figures 1, 2, 3, 4.

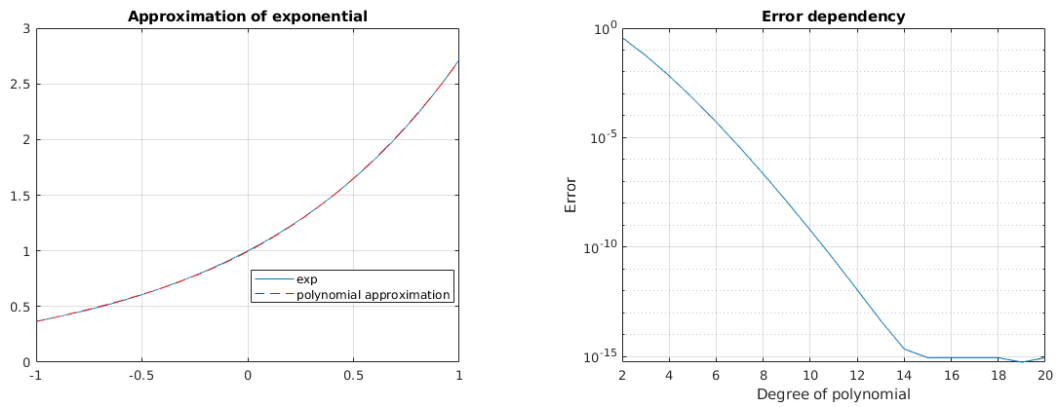
3.4 Problems with polynomial interpolation

Consider now the following function:

$$f(x) = \frac{1}{1+x^2}, \quad x \in [-5, 5].$$

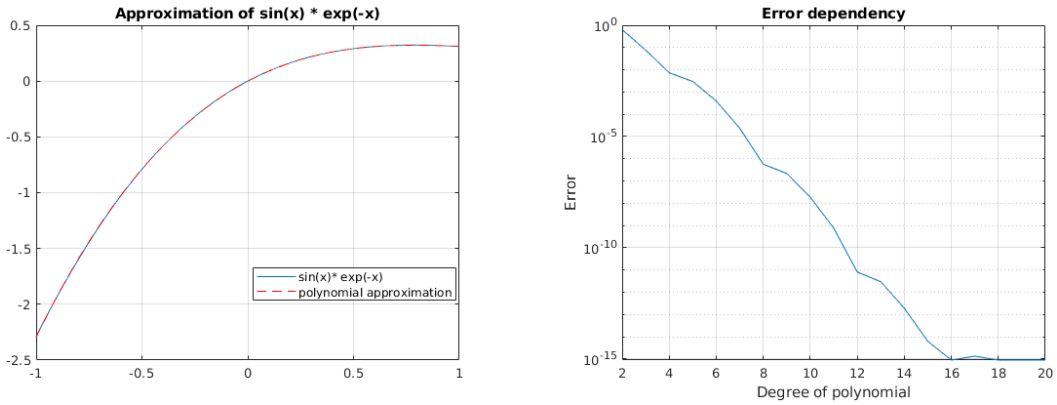
This function is known as Runge's function and it illustrates that polynomial interpolation with equidistant points can fail. If we perform a linear transformation from the interval $[-5, 5]$ to $[-1, 1]$, we obtain the equivalent function:

$$f(x) = \frac{1}{1+25x^2}, \quad x \in [-1, 1].$$



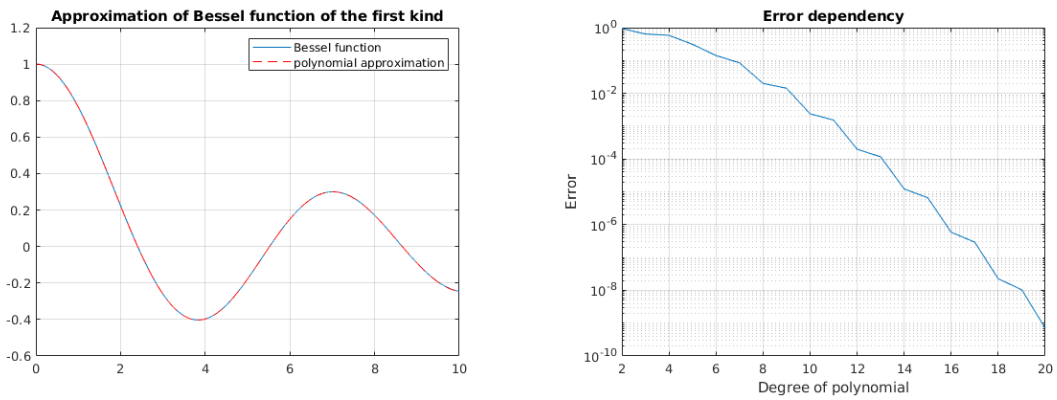
(a) Approximation using $p \in \Pi_3$ (b) Error dependence on n

Figure 2: Polynomial interpolation of $f(x) = e^x$



(a) Approximation using $p \in \Pi_3$ (b) Error dependence on n

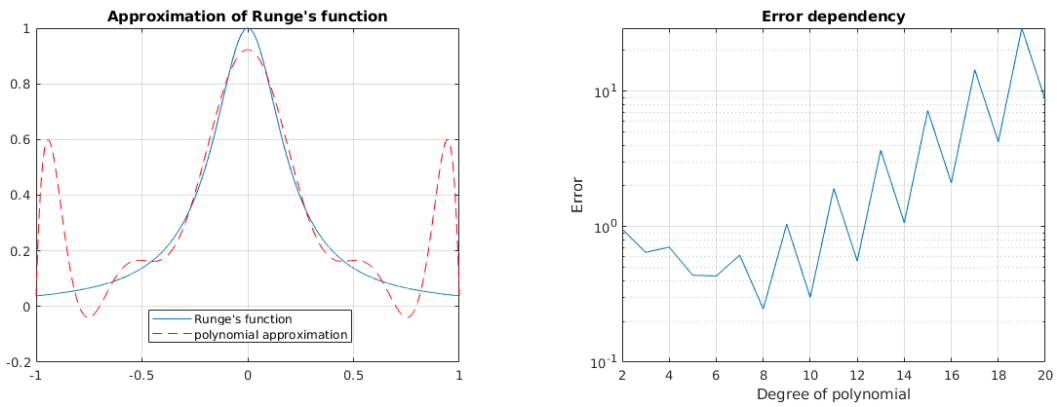
Figure 3: Polynomial interpolation of $f(x) = e^{-x} \sin x$



(a) Approximation using $p \in \Pi_10$ (b) Error dependence on n

Figure 4: Polynomial interpolation of Bessel's function of the first kind

The equivalent of Runge's function on $[1, 1]$ is plotted in Figure 5. From Figure 5a, we notice that oscillations occur towards the edges of the interval. Additionally, from Figure 5b, we observe that the error increases with the number of equidistant interpolation points, or, in other words, with the degree of the polynomial.



(a) Approximation using $p \in \Pi_1 1$ (b) Error dependence on n

Figure 5: Polynomial interpolation on equidistant nodes of Runge's function

We generalise this phenomenon and state the following important result:

Theorem 3.3 (Faber's Theorem (Kincaid and Cheney 2009))**.** For any prescribed system of nodes

$$a \leqslant x_0^{(n)} < x_1^{(n)} < \cdots < x_n^{(n)} \leqslant b \quad (n \geqslant 0)$$

there exists a continuous function f on $[a, b]$ such that the interpolating polynomials for f using these nodes fail to converge uniformly to f.

In this particular instance, we saw that, using equidistant points, the sequence of interpolating polynomials fails to converge to Runge's function. However, if we replace the interpolation nodes with Chebyshev nodes, which are the roots of the Chebyshev polynomials (ibid.), we obtain a different result. This is illustrated in Figure 6. Particularly, in Figure 6a we observe that the polynomial approximates Runge's function fairly well, while from Figure 6b we notice that the error decreases as the degree of the approximating polynomial increases.

We generalise this remark and state the following positive result:

Theorem 3.4 (Theorem on Convergence of Interpolants (ibid.))**.** If f is a continuous function on $[a, b]$, then there is a system of nodes:

$$a \leqslant x_0^{(n)} < x_1^{(n)} < \cdots < x_n^{(n)} \leqslant b \quad (n \geqslant 0)$$

such that the polynomials p_n of interpolation to f at these nodes satisfy:

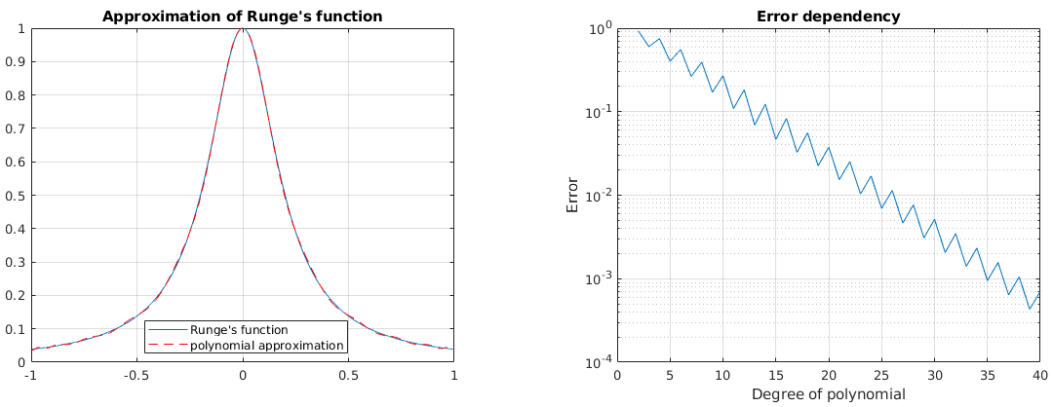
$$\lim_{n\to\infty} ||f - p_n||_\infty = 0$$

where $||f - p_n|| = \max_{x\in[a,b]} |f(x) - p_n(x)|$.

Note that this theorem only confirms the existence of a system of nodes, but much like Weierstrass' Theorem, it does not provide additional information about the location of the interpolation nodes or whether the system is unique or not.

3.5 Least-squares fitting

In the polynomial interpolation approach, given $n + 1$ data points, we construct a polynomial interpolant of degree n. This becomes problematic when we have a large number of interpolation nodes, which is common when trying to approximate functions. To avoid this problem, we would like to construct a polynomial of a moderate degree such that $\|f - p\|_\infty$ is as small as possible.



(a) Approximation using $p \in \Pi_2 4$

(b) Error dependence on n

Figure 6: Polynomial interpolation on Chebyshev nodes of Runge's function

In other words, given n data points, we want to construct a polynomial of degree m, with $m << n$, such that the error between the function and the approximation is minimised. The problem can be formulated mathematically as follows:

$$\min_{\underline{a} \in \mathbb{R}^{m+1}} \sum_{i=1}^{n} |f(x_i) - p_m(x_i)|$$

where $\underline{a} \in \mathbb{R}^{m+1}$ is a vector containing the parameters of the polynomial p_m, f is the approximated function and x_i, $i = 1, \ldots, n$ are the data points. The polynomial can be constructed using any basis.

Remark If $n = m$, the polynomial is the polynomial interpolant discussed in Section 3.3.

The problem is solved by assuming that the sum achieves its minimum for some polynomial, which means that the derivative with respect to each of the polynomial's parameters is 0. This yields a system which can then be solved. We will skip the details as it exceeds the scope of this report. However, it is important to mention the Least-squares method because it is nicely connected to the Neural Network approach, as we will see in the following section.

4 Neural Networks

Consider now the following function:

$$N : \mathbb{R} \to \mathbb{R}, \quad N(x) = \sum_{i=0}^{i=n} a_i \phi_i(x).$$

Notice that N is a linear combination. If we consider the set $\{\phi_i\}_{i=0}^{i=n}$ and set $\phi_0 = 1$, for some functions ϕ_i, the function becomes:

$$N(x) = \sum_{i=0}^{i=n} a_i \phi_i(x) = a_0 \phi_0 + \sum_{i=1}^{i=n} a_i \phi_i(x) = a_0 + \sum_{i=1}^{i=n} a_i \phi_i(x) \tag{4.1}$$

The function N defines a particular type of neural network, namely a shallow neural network, where a_0 is generally denoted by b and represents the bias of the network, the parameters a_i are denoted by w_i and represent the weights of the network and ϕ is the activation function of the network, assumed to act entrywise on its input.

Although equation 4.1 gives a mathematical description of a neural network with one hidden layer, we continue this section with formally defining artificial neural networks and their components. To better understand this, consider figure 7.

Formally, an Artificial Neural Network is a computational model based on the human nervous system, in particular on the human brain. Such systems perform tasks by learning from different examples. For instance, in the field of image recognition, a neural network can be trained to identify cars by looking at different images labelled as 'cars' or 'no cars'.

Each connected unit of the network is called a neuron, which similarly models the biological neuron. Neurons are further clustered into layers, where the first layer is called the input layer, the last layer is called the output layer and the middle layers are called hidden layers. If a neural network has only one hidden layer, then it is called a Shallow Neural Network, while if the number of hidden layers is greater than one, it is called a Deep Neural Network.

The neurons are connected by edges, more commonly knows as weights, and each neuron has a bias. The weights and biases of a network are organised in matrix or vector form. Suppose we have a neural network with L layers and that each layer has n_i neurons. Then, n_1 is the number of neurons in the input layer, which also represents the dimension of the input, and n_L is the number of neurons in the output later, or equivalently, the dimension of the output.

Therefore, a neural network is a function, say N, such that $N : \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$. At each layer i, $2 \leqslant i \leqslant L$, w_i is the matrix of weights, so that $w_i \in \mathbb{R}^{n_i \times n_{i-1}}$, and b_i is the vector of biases, so that $b_i \in \mathbb{R}^{n_i}$. Note that this is in accordance with equation 4.1 for shallow neural networks with n neurons in the hidden layer, $n_1 = 1$ and $n_3 = 1$. In particular, $n_1 = 1$ denotes that the function we are trying to approximate is of exactly one variable. This further implies that if we are trying to approximate a function of d variables, where $d \in \mathbb{N}$, then $n_1 = d$ and $n_L = 1$, as the output of a function is always a real number.

Further, we define the weighted input at layer i by:

$$z_i = w_i \phi_{i-1} + b_i, \quad z_i \in \mathbb{R}^{n_i}, \quad 2 \leqslant i \leqslant L$$

where x is the input of the network. We also define the output at layer i by:

$$\phi_i = \phi(z_i), \quad a_i \in \mathbb{R}^{n_i}, \quad 1 \leqslant i \leqslant L-1 \text{ and } \phi_1 = x \in \mathbb{R}^{n_1}.$$

The last and most important component of a neural network is the activation function, which is used to define the output of a certain layer. Commonly used

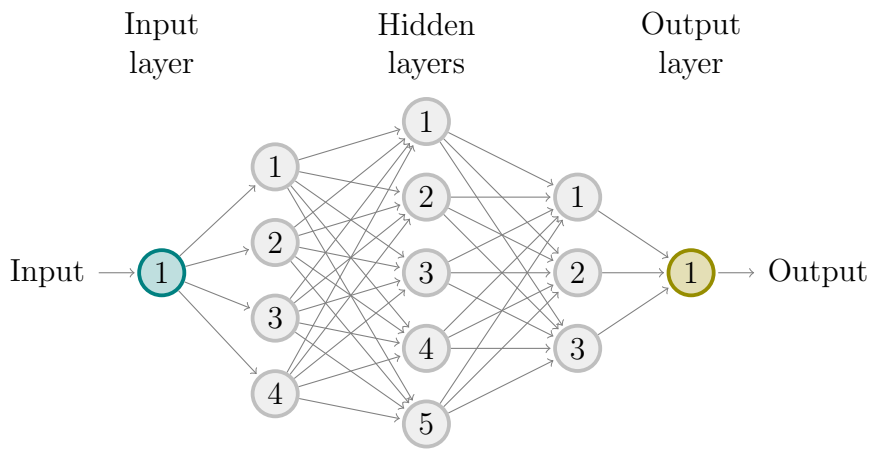


Figure 7: Deep Neural Network with 5 layers.

activation functions are the Rectified Linear Unit, given by $\phi(x) = \max\{0, x\}$ and the logistic sigmoid activation function $\phi(x) = \frac{1}{1+e^{-x}}$. Notice that the latter is a non-linear smooth, continuous, infinitely differentiable function, between 0 and 1. Other common activation functions are:

- identity function: $f(x) = x$;
- absolute value function: $f(x) = |x|$;
- cubic function, $f(x) = \max\{0, x^3\}$;
- hyperbolic tangent function, $f(x) = \max\{0, \tanh(x)\}$;
- softmax function, $f(x) = \log(1 + e^x)$.

Usually, when training neural networks, the final weighted input z_L is activated as well. However, when using neural networks to approximate functions, the final activation is skipped. Hence, the function defined by the neural network is just a linear combination of activation functions, as given in 4.1.

To better understand this, in Figure 7, we provide an example of a neural network with 5 layers (3 hidden layers), hence $L = 5$. In this particular example, we have:

$$n_1 = 1,\ n_2 = 4,\ n_3 = 5,\ n_4 = 3,\ n_5 = 1$$
$$w_2 \in \mathbb{R}^{4\times 1},\ w_3 \in \mathbb{R}^{5\times 4},\ w_4 \in \mathbb{R}^{3\times 5},\ w_5 \in \mathbb{R}^{1\times 3}$$
$$b_2 \in \mathbb{R}^4,\ b_3 \in \mathbb{R}^5,\ b_4 \in \mathbb{R}^3,\ b_5 \in \mathbb{R}^1$$
$$z_2 = w_2\phi_1 + b_2 \in \mathbb{R}^4,\ z_3 = w_3\phi_2 + b_3 \in \mathbb{R}^5,\ z_4 = w_4\phi_3 + b_4 \in \mathbb{R}^3,\ z_5 = w_5\phi_4 + b_5 \in \mathbb{R}$$
$$\phi_1 = x \in \mathbb{R},\ \phi_2 = \phi(z_2) \in \mathbb{R}^4,\ \phi_3 = \phi(z_3) \in \mathbb{R}^5 \text{ and } \phi_4 = \phi(z_4) \in \mathbb{R}^3.$$

where ϕ is the activation function.

We now need to define an appropriate learning task for the neural network. In this case words, 'learning' the behaviour of a function f implies finding values for the parameters (weights and biases) of N such that the error between f and N is minimum, ideally 0. To this purpose, we use a cost function.

Generally, when training neural networks, cost functions quantify the error between the predicted output of the network and the expected output. The cost function is the average of the loss functions for all the training examples, where the loss function computes the error for a single training example.

In the setting of our problem, to minimise the error between f and N, we first choose a set of training points in an interval $[a, b]$ where f and N are defined, compute the loss at each training example and minimise the average loss, which is the cost function. A common cost function used in training neural networks is the Mean Squared Error, defined by:

$$C(\underline{w}, \underline{b}) = \frac{1}{N} \sum_{i=1}^{i=N} \frac{1}{2} ||f(x_i) - N(x_i)||^2$$

where, $\underline{w}$ and $\underline{b}$ are the weights and biases of the network, N is the number of training points. Note that the Cost function is a function of the weights and biases of the network, since the training points are fixed. The loss function, which is a function of x, is respectively given by:

$$C_{x_i} = \frac{1}{2} ||f(x_i) - N(x_i)||^2, \quad i = 1, \ldots, N$$

Here, we use the shorthand notation $N(x)$ for the function defining the neural network to mean:

$$N(x) := N(x, \underline{w}^*, \underline{b}^*) \text{ where } (\underline{w}^*, \underline{b}^*) = \underset{(\underline{w}, \underline{b})}{\arg\min}\, C(\underline{w}, \underline{b}).$$

One thing which is useful to know when comparing the efficiency of neural network approximation and polynomial approximation is the number of parameters used. In the latter case, a polynomial of degree n has $n + 1$ parameters. For functions of one variable, a shallow neural network with n neurons in the hidden layer has $3n+1$ parameters. That is because the input and output layers have one neuron, so $w_2 \in \mathbb{R}^{n \times 1}$, $w_3 \in \mathbb{R}^{1 \times n}$, $b_2 \in \mathbb{R}^n$ and $b_3 \in \mathbb{R}$.

Before we dive into the details of how neural networks can be used to approximate functions, it is worth mentioning this remarkable result from Cybenko's paper (Cybenko 1989), which is of interest for our purpose.

Theorem 4.1 (Cybenko's Theorem (ibid.))**.** Let ϕ be any continuous sigmoidal function. Then, given any $f : [a, b] \to \mathbb{R}$ such that f is continuous on $[a, b]$ and any $\epsilon > 0$, there exists a sum $N(x)$, of the form:

$$N(x) = \textstyle\sum_{i=1}^{i=n} a_i \phi(z)$$

such that $||f - N||_\infty < \epsilon$, where $z = wx + b$.

Notice that the function N defines an artificial neural network with only one hidden layer. Then, just as Weierestrass' Theorem for polynomials, Cybenko's theorem states that there exists a function N describing a shallow artificial neural network, which can approximate any continuous function with arbitrarily precision when using a sigmoidal function as activation function. However, it does not give any additional information about the number of neurons in the hidden layer or whether the function N is unique or not.

In the next section we examine some numerical results from the experiments we conducted.

5 Numerical experiments

5.1 Initial guess

In order to train the network, a starting point needs to be provided, an initial guess for the weights and biases. We used multiple alternatives: zeros, ones, and random parameters. On the one hand, a known initial guess, such as zeros, ones or controlled random parameters, can help compare how other factors affect the quality of the approximation. On the other hand, using uncontrolled random initial parameters is also useful, especially when using an activation function which is sensitive to the initial guess, because it proves the importance of a good initialisation.

5.2 Optimisation

The goals of optimisation algorithms is to find a global minimum for the cost function. However, this is not an easy task and, more often than not, the algorithm finds a local minimum or a potential local minimum, rather than the global one. This is still an active area of research.

In our experiments, we mostly used the `lsqnonlin` solver available in the MATLAB optimisation toolbox. This seemed to be the most effective optimisation algorithm we tried. We treated it as a so-called 'blackbox', hence, we will not go into the details of its actual implementation here. What is worth mentioning is that MATLAB allows the user to choose the algorithm for the solver and we opted for the Levenberg-Marquardt algorithm.

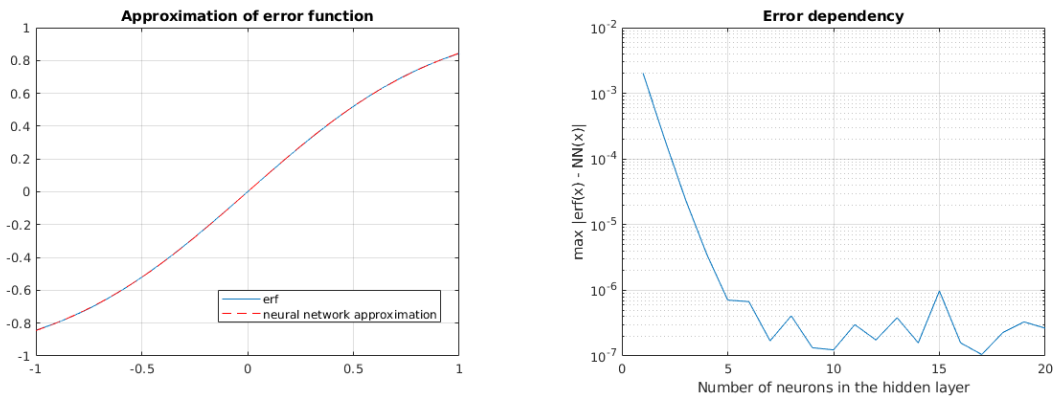
Some alternatives are the `fminunc` solver from the same toolbox and the Gradient Descent algorithm.

5.3 Numerical results in 1D

Most of the functions we considered were continuous with continuous derivatives, such as the error function and the exponential. A few polynomials were considered and a couple of non-differentiable functions. We investigated the error in the L^∞ norm. Most of the experiments conducted used the logistic sigmoid function as the activation function, since we noticed experimentally that it produces the best results. ReLU, tanh, identity, absolute value, cubic and softmax were also considered. We mostly used equidistant points as training data, but experiments using Chebyshev nodes were run as well.

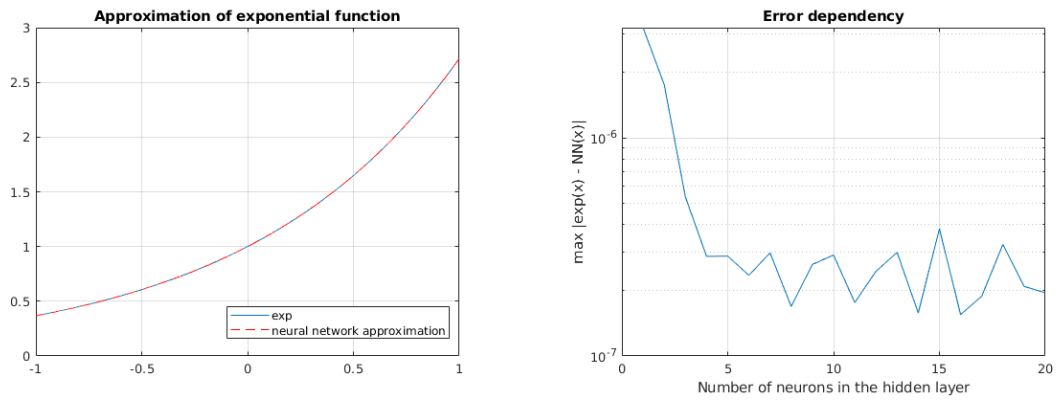
As opposed to more traditional Machine Learning problems, in function approximation data is not a problem. Hence, the graphs showing the error dependency on the number of neurons in the hidden layer below assume that sufficient data is provided. More specifically, even if more training data is fed into the network, the approximation does not improve.

While one layer is enough to get 10^{-6} precision, it is important to know the minimum number of neurons in the hidden layer to obtain a possible approximation. Since a neural network function is linear combination of activation functions, in this case logistic sigmoid, every neuron essentially represents one term in the linear combination. Hence, it seems intuitive that if the derivative of the approximated function changes sign, say, N times, $N + 1$ neurons will be needed in the hidden layer.



(a) Approximation using $n = 1$ (b) Error dependence on n

Figure 8: Shallow neural network approximation of error function



(a) Approximation using $n = 1$ (b) Error dependence on n

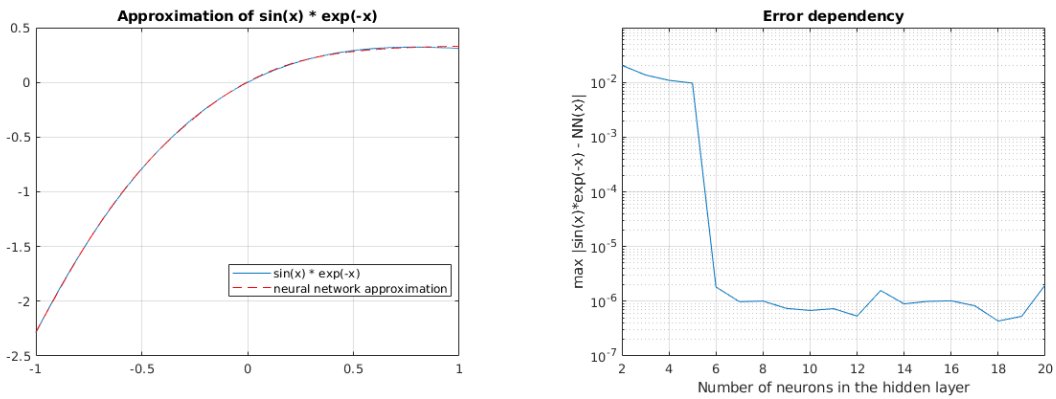
Figure 9: Shallow neural network approximation of $f(x) = e^x$

Consider the function $f(x) = e^{-x} \sin x$. Its derivative changes sign at $\frac{\pi}{4}$, so two neurons are needed in hidden layer, as illustrated in Figure 10. On the other hand, the derivatives of erf and e^x do not change sign, therefore one neuron in the hidden layer yields a good enough approximation, as shown in Figures 8 and 9.

The first function we examined was the error function. In Figure 8, we plot the approximation obtained using one neuron in the hidden layer, or equivalently 4 parameters, and the error dependence on the number of neurons in the hidden layer. Notice the rapid decay from one neuron to five in the hidden layer.

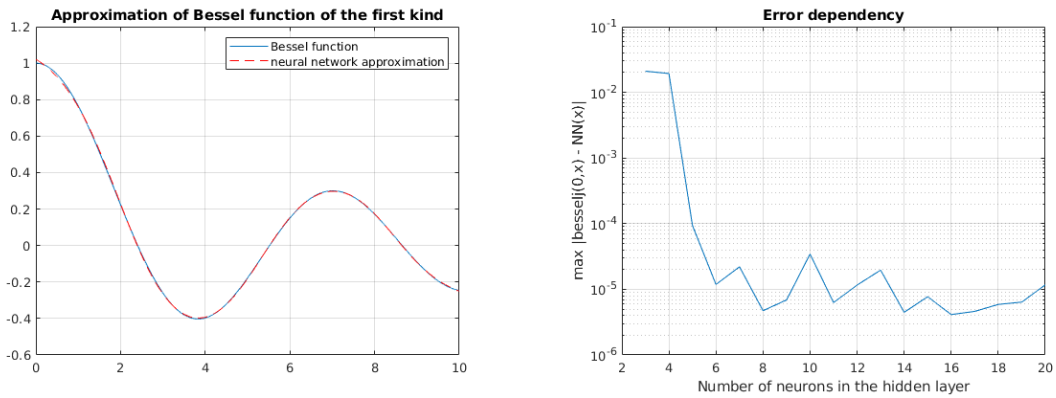
For comparison, we include the results obtained for $f(x) = xe^x$, $f(x) = e^{-x} \sin x$ and Bessel's function of the first kind in Figures 9, 10 and 11, using equidistant nodes and the logistic sigmoid function as activation function.

But more interestingly, in Figures 12 and 13 we plot the neural network approximation of Runge's function on both equidistant and Chebyshev nodes. From these graphs, we observe that the neural network approach does not suffer from Runge's phenomenon.



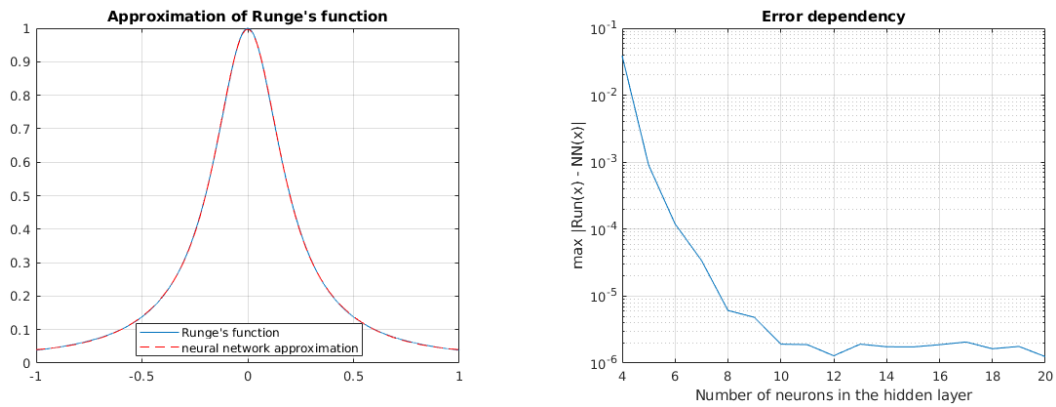
(a) Approximation using $n = 2$ (b) Error dependence on n

Figure 10: Shallow neural network approximation of $f(x) = e^{-x} \sin x$



(a) Approximation using $n = 3$ (b) Error dependence on n

Figure 11: Shallow neural network approximation of Bessel's function (1st kind)

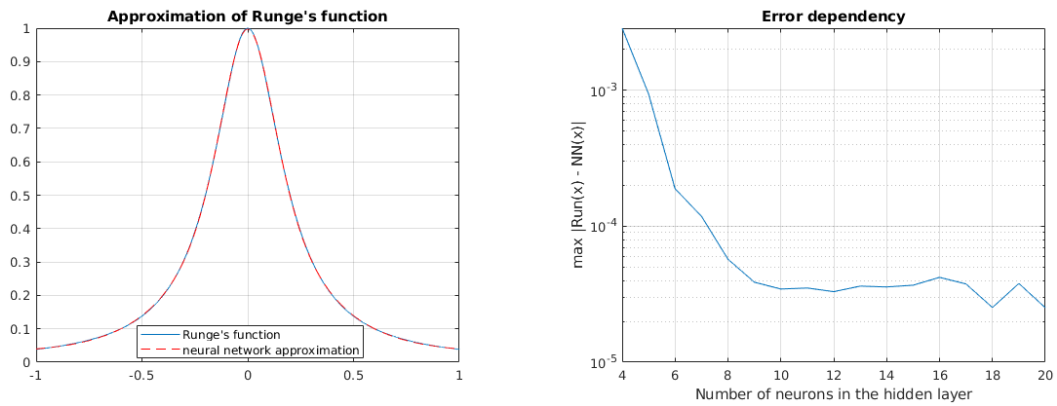


(a) Approximation using $n = 4$ (b) Error dependence on n

Figure 12: Neural network approximation on equidistant nodes of Runge's function

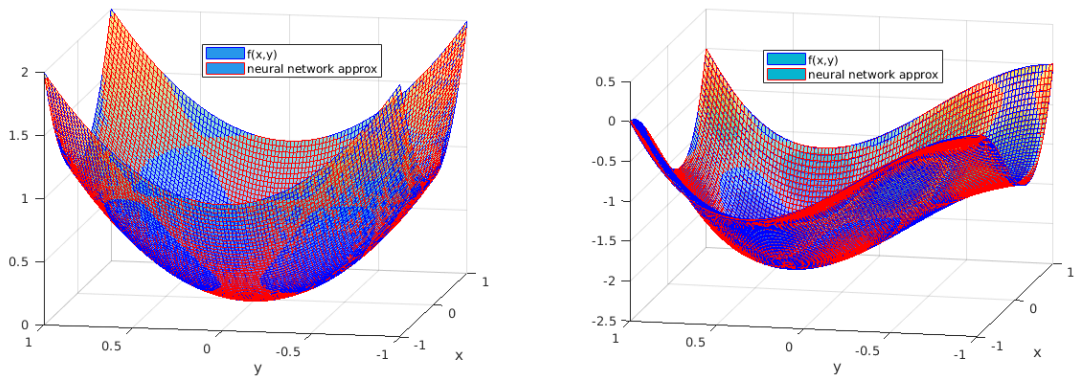
5.4 Numerical results in multiple dimensions

To move from one dimension to d dimensions, the main modification in the structure of the neural network is the number of neurons in the input layer. In particular, for a function of d variables, d neurons in the input layer are needed.



(a) Approximation using $n = 4$ (b) Error dependency on n

Figure 13: Neural network approximation on Chebyshev nodes of Runge's function



(a) Approximation of $x^2 + y^2$ using $n = 4$ (b) Approximation of $f(x)$[1]using $n = 4$

Figure 14: Functions of two variables shallow neural network approximation

A fairly good approximation can be achieved using just one hidden layer, where, similar to the one-dimensional case, the number of neurons in the hidden layer depends on how many times the partial derivatives change sign. It seems that the approximation can be improved by adding hidden layers to the network. However, we experimented very little with these functions as the training is very slow.

Some results are presented in Figure 14. The function and its neural network approximation can barely be distinguished.

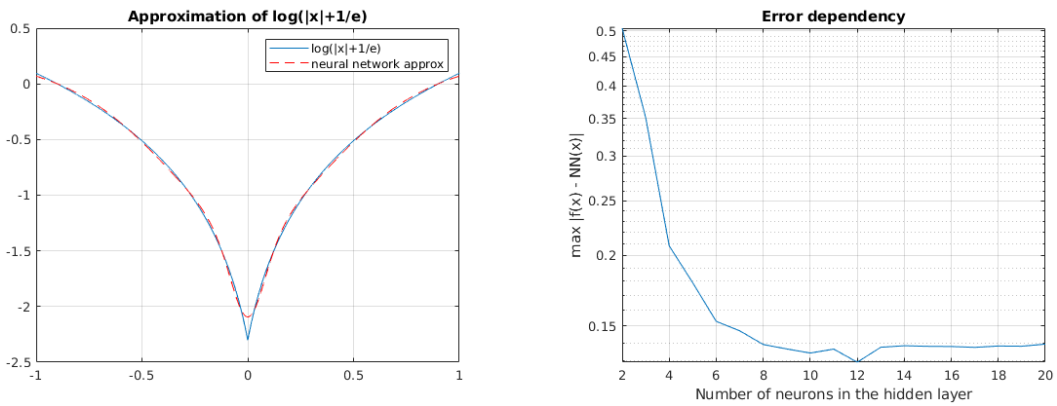
5.5 Problems with Neural Network Approximation

As mentioned in the previous section, the neural network approach has some advantages over other numerical approaches. But this method also has its drawbacks.

To begin with, not all functions can be well approximated. Consider, for example, $f(x) = \log(|x| + \frac{1}{e})$, which is continuous, but not differentiable at 0. As illustrated in Figure 15, the approximation improves as the number of neuron in the hidden layer increases, but the error around 0 is still notably large.

Additionally, if the derivative of the function being approximated changes sign many times, more neurons, hence more parameters are needed. From our experi-

[1] $f(x) = x^3 + x^2 - x + y^3 + y^2 - y - 2$



(a) Approximation using $n = 4$

(b) Error dependence on n

Figure 15: Shallow neural network approximation of $\log(|x| + \frac{1}{e})$

ments, it seems that the optimisation routine performs naturally worse as the number of variables involved increases. This also implies that it can get computationally expensive, especially as the number of dimensions increases.

Finally, another big disadvantage of this implementation is that there are many unknowns. Why does one activation function yield a better approximation than another? What is a good initialization of the parameters? What is the ideal number of training nodes? Is this method stable and is there convergence? The only thing we can say is that no matter how we change these variables, the error does not usually go below the 10^{-6} bound.

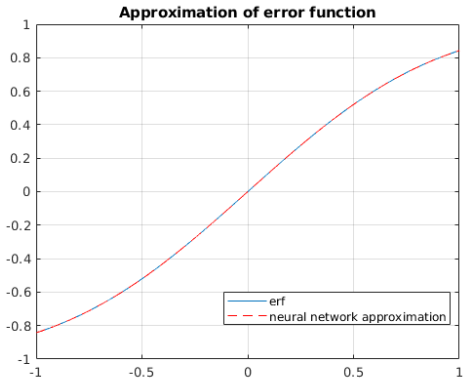
5.6 Numerical results - Piecewise Neural Networks

A different approach to function approximation is piecewise polynomial interpolation. This technique is very similar to the polynomial interpolation method, described in Section 3.3. The key difference is that, in order to avoid using a large number of nodes, or equivalently a large degree of the interpolating polynomial, we can split the interpolation interval in m small subintervals and approximate over each piece by a polynomial of low degree (for example. $p \in \Pi_1$ or $p \in \Pi_2$).

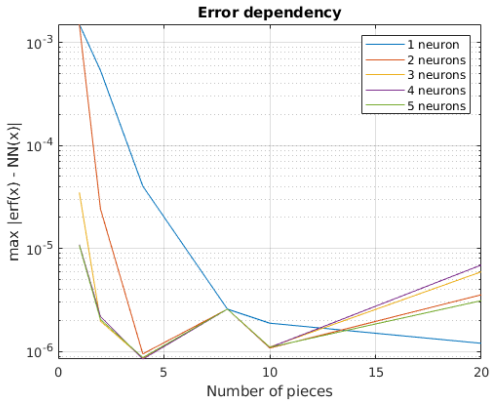
Inspired by this idea, we tried to decrease the error below 10^{-6} by using a piecewise shallow neural network approximation. We investigated the same functions as in Section 5.3 and the results are presented below, where m is the number of subintervals and n is the number of neurons in the hidden layer.

Here, we used the interval $[-1, 1]$, except for Bessel's function of first kind where we used $[0, 10]$, which we then split in 1, 2, 4, 8, 10 and 20 subintervals.

There are a few observations which can be drawn from these graphs. Firstly, there's no clear correlation between the error and the number m of subintervals used. Secondly, more neurons on small pieces sometimes yield a worse approximation as opposed to using just one neuron in the hidden layer. And finally, we still could not decrease the error below 10^{-6}.

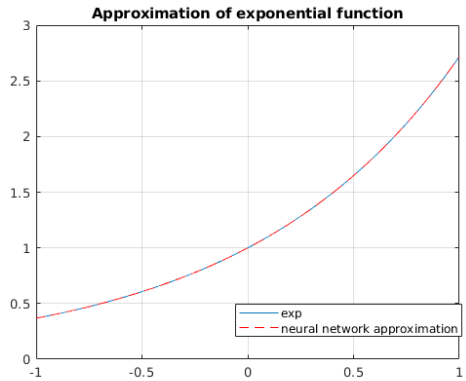


(a) Approximation using $m = 10$ and $n = 1$

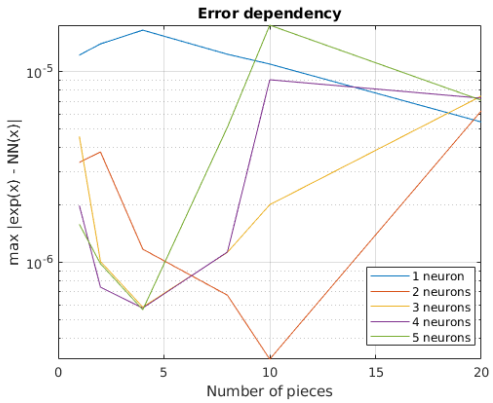


(b) Error dependence on m and n

Figure 16: Piecewise neural network approximation of error function

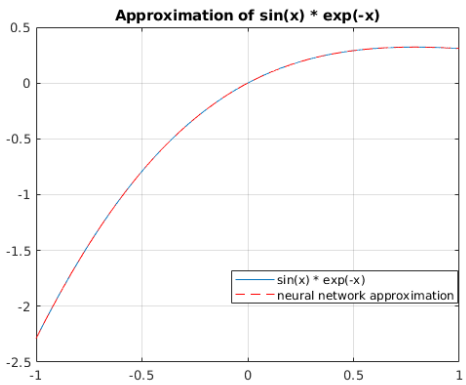


(a) Approximation using $m = 10$ and $n = 1$

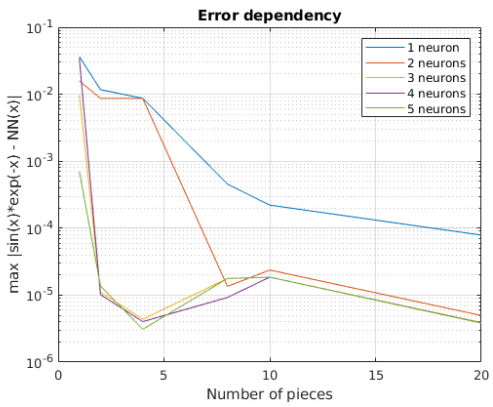


(b) Error dependence on m and n

Figure 17: Piecewise neural network approximation of $f(x) = e^x$

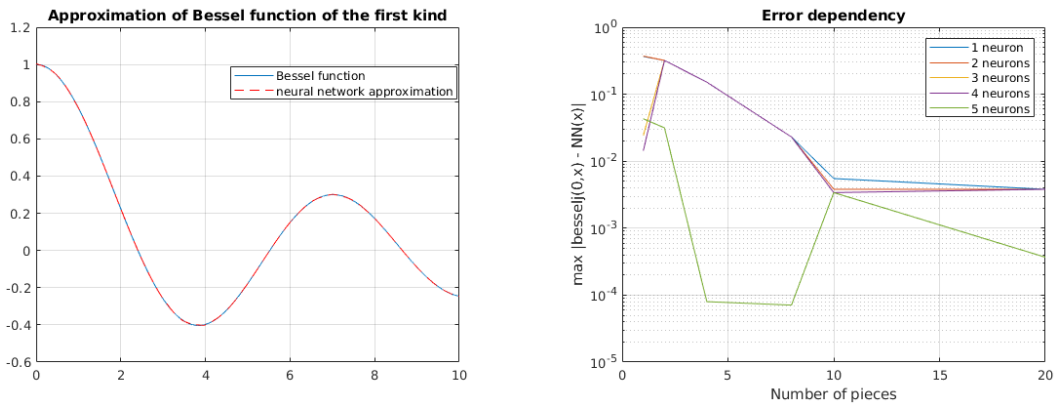


(a) Approximation using $m = 10$ and $n = 1$



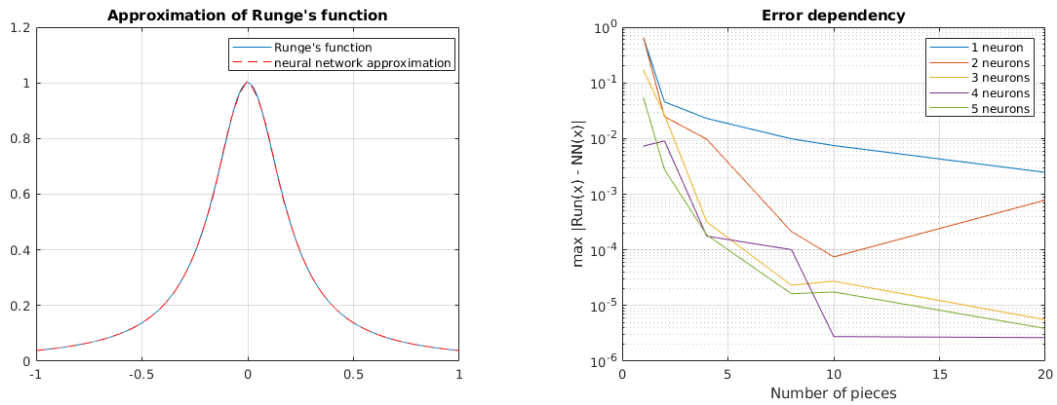
(b) Error dependence on m and n

Figure 18: Piecewise neural network approximation of $f(x) = e^{-x} \sin x$



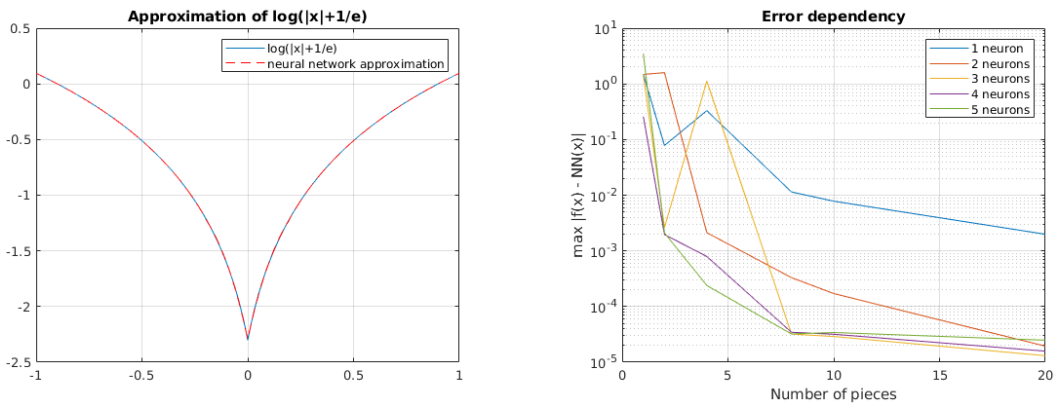
(a) Approximation using $m = 10$ and $n = 1$ (b) Error dependence on m and n

Figure 19: Piecewise neural network approximation of Bessel's function (1st kind)



(a) Approximation using $m = 10$ and $n = 1$ (b) Error dependence on m and n

Figure 20: Piecewise neural network approximation of Runge's function



(a) Approximation using $m = 10$ and $n = 1$ (b) Error dependence on m and n

Figure 21: Piecewise neural network approximation of $f(x) = \log(|x| + \frac{1}{e})$

5.7 Problems with piecewise approach

Let us first point out that this method has its advantages. First of all, as illustrated in Figure 21, the piecewise neural network approach does not require the approximating function to be differentiable. Furthermore, if the subintervals are small enough, each piece can be fairly well approximated using just one neuron in the hidden layer. In other words, the number of parameters needed is smaller than in the simple neural network approximation.

Unfortunately, there is still one major disadvantage. As with the one-piece approach, there are many unknowns. One could say there are even more, because, as observed from the figures above, more smaller subintervals does not necessary yield a better approximation. That is, the ideal number of subintervals and whether they should be equal or not cannot be established.

Moreover, it is true that this method can get computationally expensive. However, if it generates better results than the simple neural network approximation, the training can be done in parallel. This is less expensive and definitely faster.

Finally, when trying to approximate $f(x) = x$ using the Rectified Linear Unit as an activation function, using a proper initialisation for the parameters, the error obtained was near 10^{-13}. When using very small subintervals, the approximating function is approximately equal to $mx+n$. Combining these two results, we believed that if the original interval is subdivided into very narrow subintervals, then any function could be approximated up to 10^{-13} precision using ReLU. However, the experiments showed that this is not the case, which is even a more problematic disadvantage of the piecewise neural network approach.

With this in mind, we now consider an application of function approximation. In the following section, we briefly cover how deep neural networks can be employed to solve boundary value problems.

6 Boundary Value Problems

6.1 The Variational and Weak Formulations

The particular type of boundary value problems we are interested in is:

$$\begin{aligned} -\Delta u(\underline{x}) &= f, \quad \underline{x} \in \Omega \\ u(\underline{x}) &= 0, \quad \underline{x} \in \partial\Omega \end{aligned} \tag{6.1}$$

where $x \in \mathbb{R}^d$ is a vector, Ω is a domain in $\mathbb{R}^d$, $\partial\Omega$ is the boundary of the domain Ω and f is some continuous function.

In Johnson (2009), the solution to 6.1 is shown to also be the solution to a variational problem and a minimization problem.

To formulate the variational and minimization problems, consider $n = 1$ in 6.1, the linear space V, given by:

$$V = \{v : v \text{ is a continuous function on } \Omega,\ v' \text{ is piecewise continuous and bounded on } \Omega, \text{ and } v(x) = 0, \forall x \in \partial\Omega\}$$

and the linear functional $F\colon V \to \mathbb{R}$, given by:

$$F(v) = \frac{1}{2}(v', v') - (f, v)$$

where

$$(v, w) = \int_0^1 v(x)w(x)dx.$$

The variational problem is then defined as:

$$\text{Find } u \in V \text{ such that } (u', v') = (f, v),\ \forall v \in V$$

and the minimization problem is defined as:

$$\text{Find } u \in V \text{ such that } F(u) \leqslant F(v),\ \forall v \in V$$

We will not give the proof of how the solution to 6.1 is also the solution to these problems. What is of interest, however, is that we can now derive the weak formulation:

$$\min_{u \in V} I(u), \tag{6.2}$$

where

$$I(u) = \int_\Omega \Big(\frac{1}{2}|\nabla u(x)|^2 - f(x)u(x)\Big)dx$$

Here, we choose V to be the Sobolev space, $H_0^1(\Omega)$ and $u \in V$ is a trial function. In optimization terms, $I(u)$ is the merit or objective function. This formulation is particularly useful because solutions to non-linear boundary value problems can be obtained by slightly modifying $I(u)$, as we show later in the section.

6.2 Trial Functions and Neural Network Training

The trial function and the structure of the neural network we used are the ones defined in E and Yu (2018).

Both the hidden layers and the input layer have the same number of neurons m. The key idea is to define a linear transformation which approximates the trial

function u such that $x \to z_\theta(x) \in \mathbb{R}^m$. Then, the architecture comprises the usual input and output layers, with the remaining (deep) layers consisting of several blocks, which in turn incorporate two linear transformations, two activation functions, and a residual connection. Hence, the ith block can be defined as:

$$f_i(s) = \phi(w_{i,2} \cdot \phi(w_{i,1} \cdot s + b_{i,1}) + b_{i,2}) + s$$

where $s \in \mathbb{R}^m$ is the input, $w_{i,1}$, $w_{i,2} \in \mathbb{R}^{m \times m}$, $b_{i,1}$, $b_{i,2} \in \mathbb{R}^m$ are the parameters of the block and ϕ is the activation function.

Note that if we are solving a boundary value problem in d dimensions, where usually $d \neq m$, there are two options:

- $d < m$, in which case the paper suggests padding the vector s with $m - d$ zeros;
- $d > m$, in which case a linear transformation should be applied such that $x \in \mathbb{R}^d \to s \in \mathbb{R}^m$.

Having defined s and f_i, $i = 0, \ldots, n$, the full network can be described as:

$$z_\theta(x) = f_n \circ \cdots \circ f_1(x),$$

where θ is the set of all parameters. Finally, u can be defined as:

$$u(\theta; x) = a \cdot z_\theta(x) + b.$$

We can include a and b in θ as well, as they are also unknowns. Therefore, the minimization problem 6.2 becomes:

$$\min_\theta \int_\Omega \Big(\frac{1}{2} |\nabla_x u(\theta; x)|^2 - f(x) u(\theta; x) \Big) dx. \tag{6.3}$$

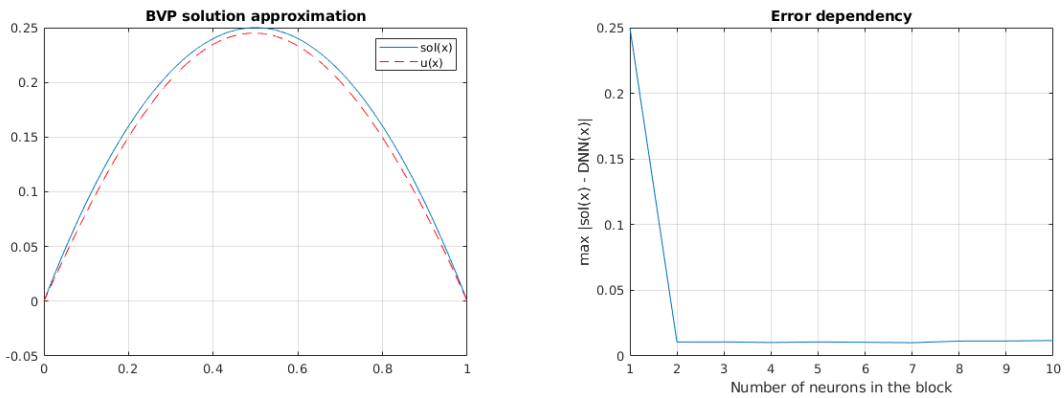
As stated in the E and Yu (ibid.), approximating this integral fits nicely with the Gradient Descent algorithm. However, note that no boundary conditions were imposed anywhere in the definition of the trial function. So solving the minimization problem directly will in fact give any function that satisfies the differential equation, which is not unique. To deal with this problem, the paper includes a penalty term which satisfies the boundary conditions and which, again, fits nicely with the Gradient Descent optimization.

In out experiments, we used a different approach. Instead of solving an unconstrained minimization problem with a regularization term, we solved a constrained minimization problem.

6.3 Numerical Results

As solving a boundary value problem is a particular application of function approximation, we ran into some of the same problems from Section 5. In particular, initialising the parameters proved to be very important and somehow problematic. To solve the minimisation problem 6.3 subject to boundary conditions, we used the `fmincon` routine from the MATLAB Optimization Toolbox. In particular, we opted for the Interior Point algorithm.

Most of the boundary value problems we examined were one-dimensional, as we are yet unable to obtain satisfactory results in multiple dimensions. We split the interval in equidistant nodes and used the logistic sigmoid as the activation function for the network. Note that in E and Yu (ibid.), problems in as far as 100 dimensions are considered.



(a) Approximation using $b = 1$ and $m = 2$ (b) Error dependence on m

Figure 22: Deep Neural Network solution of Boundary Value Problem 6.4

The first problem we considered was:

$$\begin{cases} -u''(x) &= 2, \quad x \in (0,1) \\ u(x) &= 0, \quad x \in \{0,1\} \end{cases} \tag{6.4}$$

where

$$I(\theta; u) = \int_0^1 \left(\frac{1}{2}|u'(\theta; x)|^2 - 2 \cdot u(\theta; x)\right)$$

The actual solution is given by:

$$u(x) = x - x^2.$$

The results are presented in Figure 22, where b represents the number of blocks in the network, m the number of neurons in each block and $sol(x) = u(x)$. Notice that the error does not improve as the number of neurons in the block increases. Similar results were obtained using different functions, such as $f(x) = 6x$, $f(x) = 12x^2$, $f(x) = 20x^3$, $f(x) = e^x$, and $f(x) = \sin x$.

Next, we investigated non-linear differential equations of the form:

$$-u''(x) + g(u) = f, \quad x \in \Omega.$$

where $I(u)$ is given by:

$$I(u) = \int_\Omega \left(\frac{1}{2}|u'(x)|^2 - f(x)u(x) + \int g(u)du\right)dx$$

For example, we solved the following boundary value problem:

$$\begin{cases} -u'' + u + u^2 + \cos(u) &= x^4 - 2x^3 + x + \cos(x - x^2) + 2, \quad x \in (0,1) \\ u(x) &= 0, \quad x \in \{0,1\} \end{cases} \tag{6.5}$$

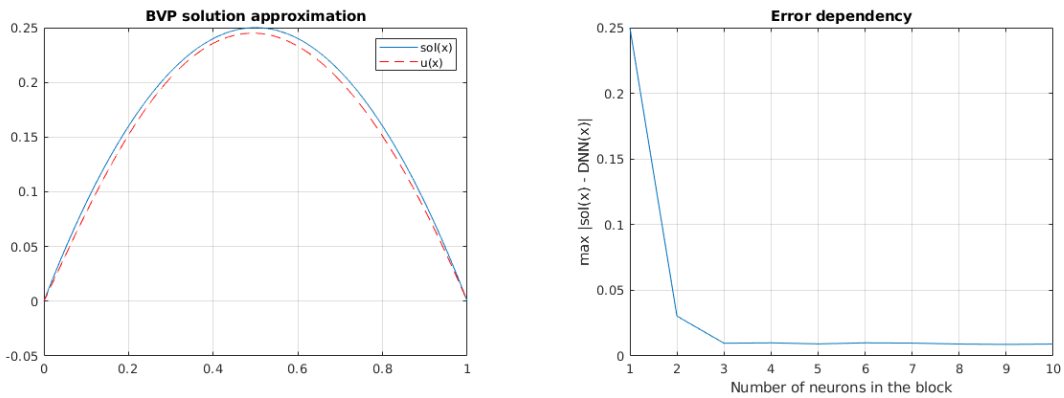
where f is chosen such that the solution is given by:

$$u(x) = x - x^2.$$

In this case, the objective function is:

$$I(u) = \int_0^1 \left(\frac{1}{2}|u'(x)|^2 - f(x)u(x) + \frac{1}{2}u(x)^2 + \frac{1}{3}u(x)^3 + \sin(u(x))\right)dx$$

The results are illustrated in Figure 23. Additionally, similar results were obtained using different choices of u.



(a) Approximation using $b = 1$ and $m = 3$ (b) Error dependence on m

Figure 23: Deep Neural Network solution of Boundary Value Problem 6.5

As was the case with function approximation, adapting from one dimension to, say, d dimensions, is as simple as adding $d - 1$ neurons in the input layer. Then, depending on the number of neurons in each block, the input vector is either padded with zeros or linearly transformed so that the dimensions match. Note that while this method can be applied to d-dimensional problems, there is still the issue of splitting the data appropriately. In other words, we can usually solve boundary value problems using this approach as long as the domain can be easily partitioned in training examples.

Moreover, there is still much to be explored concerning the structure of the neural network, the optimization algorithm used, the activation function, or the initialisation of the parameters.

7 Summary

As stated in the beginning, this report does not present any novel material. Instead our aim was to describe some of the experimental results we obtained trying to approximate various functions and solve boundary value problems.

Some of the problems we encountered represent active areas of research, such as optimization algorithms, particularly for non-linear models, data sampling, especially in high dimensions, or even the problem formulation. For example, the Deep Ritz Method is only possible because of the availability of a variational formulation of the boundary value problem. Perhaps a different construction or model would yield an even better approach.

Moreover, there are theoretical results (Barron 1993) which state and prove that there exists a connection between the number of neurons in the hidden layer and the absolute error between the function and its approximation when implementing the Shallow Neural Network approach we presented. However, experimentally we were not able to verify these results, mostly due to the limitations of our nonlinear optimisation solvers.

Finally, even though some of the results we emphasized are negative, neural networks, deep learning in particular, provide a very efficient solution to some of the problems in approximation theory (Sun, Yan, and Sclabassi 2003; Lagaris, Likas, and Papageorgiou 2000; Rudd and Ferrari 2015). They have their advantages and disadvantages, as do other numerical methods. Nonetheless, it is a very exciting area of research and it certainly raises many interesting questions.

References

Barron, A. R. (1993). "Universal approximation bounds for superpositions of a sigmoidal function". In: *IEEE Transactions on Information Theory* 39.3, pp. 930–945. DOI: 10.1109/18.256500.

Cheney, E. W. (1966). *Introduction to Approximation Theory*. United States of America: McGraw-Hill.

Cybenko, G. (1989). "Approximation by Superpositions of a Sigmoidal Function". In: *Mathematics of Control Signals and Systems* 2, pp. 303–314. DOI: https://doi.org/10.1007/BF02551274.

E, W. and B. Yu (2018). "The Deep Ritz Method [A Deep Learning-Based Numerical Algorithm for Solving Variational Problems]". In: *Communications in Mathematics and Statistics* 6.1. DOI: https://doi.org/10.1007/s40304-018-0127-z.

Johnson, C. (2009). *Numerical Solution of Partial Differential Equations by the Finite Element Method*. Dover Publications Inc.

Kincaid, D. and E. W. Cheney (2009). *Numerical Analysis: Mathematics of Scientific Computing*. 3rd edn. United States of America: American Mathematical Society.

Lagaris, I. E., A. C. Likas, and D. G. Papageorgiou (2000). "Neural-network methods for boundary value problems with irregular boundaries". In: *IEEE Transactions on Neural Networks* 11.5, pp. 1041–1049. DOI: 10.1109/72.870037.

Mazumder, S. (2015). *Numerical Methods for Partial Differential Equations: Finite Difference and Finite Volume Methods*. Academic Press.

Morton, K. W. and D. F. Mayers (2005). *Numerical Solution of Partial Differential Equations: An Introduction*. 2nd edn. Cambridge University Press.

Rudd, K. and S. Ferrari (2015). "A constrained integration (CINT) approach to solving partial differential equations using artificial neural networks". In: *Neurocomputing* 155, pp. 277–285. DOI: 10.1109/72.870037.

Smith, G. D. (1986). *Numerical Solution Of Partial Differential Equations: Finite Difference Methods*. 3rd edn. United States of America: Oxford University Press.

Strang, G. and G. Fix (2008). *An Analysis of the Finite Element Methods*. 2nd edn. Wellesley-Cambridge Press.

Sun, M., X. Yan, and R. J. Sclabassi (2003). "Solving partial differential equations in real-time using artificial neural network signal processing as an alternative to finite-element analysis". In: *International Conference on Neural Networks and Signal Processing*. DOI: 10.1109/ICNNSP.2003.1279289.